

# ENVIRONMENT DIAGRAM REVIEW AND INTRODUCTION TO RECURSION Meta

---

## COMPUTER SCIENCE MENTORS 61A

February 10, 2026 – February 13, 2026

### Example Timeline

- Teaching tips:

**Each problem's "Suggested Time" is not assigned with the 1 hour time for sections in mind.** It is more of a note of how long each individual problem may take. That is, if you added all the "Suggested Times" in the coding section, it could very well be greater than 1 hour (especially in more complicated worksheets in the future).

**However, the time in brackets will add up to 50 minutes, and is intended to be a suggested outline for your section.**

Also, the worksheet feedback form is linked at the bottom of this worksheet!

- Intros, Ice Breakers, & Logistics [5 min]

Introduce yourself, and give an overview of what CSM offers. Start off with an icebreaker of your choice!

- Mini-lecture: Review of Environment Diagrams, Lambda Expressions (If Needed) [5 min]
- Q1: Environment Diagram with Lambda Expressions [7 minutes]
- Mini-Lecture: Introduction to Recursion [10 min]

You can use the next question as an example, as it ties into the explanation of recursion!

- Q3: Boba Line [7 minutes]
- Q4: Gibonacci [7 minutes]
- Q5: Counting Digits [7 minutes]
- Q6: FizzBuzz [If Extra Time]

## 1 Environment Diagram Practice

---

1. Draw the environment diagram that results from running the code.

```
def a(y):  
    d = 1  
    b = lambda x: y(x)  
    e = lambda x: x(3)  
    return e(b)  
d = 5  
a(lambda x: 4 - x + d)
```

<https://goo.gl/9vxEwv>

Suggested Time: 7 minutes The goal of this problem is to show how lambda functions are represented in an environment diagram. This problem can be confusing, so you may want to draw it out step by step.

## 2 Introduction to Recursion

---

**There are three steps to writing a recursive function:**

1. Create base case(s)
2. Reduce your problem to a smaller subproblem and call your function recursively to solve the smaller subproblem(s)
3. Use the subproblems' solutions as pieces to construct a larger problem's solution (This can happen in many layers!)

### **Real World Analogy for Recursion**

Imagine that you're in line for boba, but the line is really long, so you want to know what position you're in.



You decide to ask the person in front of you how many people are in front of them. That way, you can take their response and add 1 to it to find your place. Now, the person in front of you is faced with the same problem that you were trying to solve, with one less person in front of them than you. They decide to take the same approach that you did by asking the person in front of them. This continues until the very first person in line is asked. At this point, the person at the front knows that there are 0 people in front of them, so they can tell the person behind them that there are 0 people in front. Now, the second person can figure out that there is 1 person in front of them, and can relay that back to the person behind them, and so on, until the answer reaches you.

Looking at this example, we see that we have broken down the problem of "how many people are there in front of me?" to  $1 +$  "how many people are there in front of the person in front of me"? This problem will terminate with the person at the front of the line (with 0 people in front of them). Putting this into more formal terms, we are breaking down the problem into a **recurrence relationship**, and the termination case (when the question gets to the very first person in line) is called the **base case**.

As a program goes through recursion, it doesn't formally solve the problem until it reaches the base case, in which case it works its way up from the base case to the original input to construct your final answer. Just like your question as you stand in line, the program:

1. goes out (down the call stack), person by person (call by call) (**recursive case**)
2. until the person at the very front is reached (**base case**)
3. when the answers get added onto and eventually passed back to you, solving the problem. (**constructing the final solution with solutions to subparts**)

### Teaching Tips

1. Base Case - What is the simplest case? Or in what case do you want your recursion to stop? It's helpful to use edge cases to nudge students if they get stuck.
2. Break the problem down into smaller problems (Try to address this in terms of each specific problem, then extrapolate for general understanding)
  - What do you need to do to reach your base case?
  - For example: in factorial (usually seen in lecture), we have to subtract by one each time we do a recursive call
3. Solve the smaller problem recursively
  - How would you use the solution to the smaller problem to write a solution to the original problem?
  - "Recursive Leap of Faith"—When writing the recursive statement, assume the function works as intended for the smaller problems. Trust. (Abstraction! woohoo!)
  - If you don't know what the recursive call needs to be, you can take an educated guess and see what happens.
  - It's often extremely helpful to run line by line through a doctest to test both a tentative solution and your understanding to a problem.
  - When running through a problem, it's often helpful to find a way to visualize the recursion in action! For shorter problems, one way you can do this is through drawing a stack of "boxes," each containing the result of a recursive call inside them, going all the way until the base case. Other linear visualizations work also!

We tend to throw around the term "recursive leap of faith" a lot, and I think that it confuses students. The "recursive leap of faith" is not synonymous with "the recursive call is correct". Rather it's a specific assumption we make while writing a recursive function that the recursive calls we make produce the correct output, even if we're not done writing our function. That is, the function we're writing works even if we're not done writing it. The fact that recursive calls return the correct value in the completed function is a mathematical fact that does not require any faith, so you should not conflate the two. Recursion is not magic; it is math.

The recursive leap of faith is essentially the scaffolding we need to help us build the recursive function. We pour the concrete for the base case and then layer our recursive logic on top of that until we have a sturdy structure, using the leap of faith's scaffolding to help us, as fallible human builders, to figure out the right way for the pieces to fit together. Once we're done, we can remove the scaffolding, but our tower still stands strong and sturdy.

4. Translate the example of standing in a line for boba into Python! We can represent our line as an integer of 0s and 1s, where the 0s represent an empty space and the 1s represent a person standing in line. You are about to stand at the end of the line, but first, you need to count how many people would be in front of you! Complete the definition for `boba_line`, which takes in an integer  $n$  and returns the number of 1s in it. Implement the function recursively, with a base case and a recursive case.

Hint: To get the last digit of an integer  $n$ , you can do  $n \% 10$  To remove the last digit from an integer, you can do  $n // 10$

```
def boba_line(n):
    """Takes in an integer of 0s and 1s and returns the number of 1s.

    >>> boba_line(0)
    0
    >>> boba_line(1)
    1
    >>> boba_line(10101)
    3
    >>> boba_line(10001)
    2
    """
    if _____ :                # base case

        return _____

    else:

        return _____ + _____ # recursive case

    if n < 10:
        return n
    else:
        return (n % 10) + boba_line(n // 10)
```

Suggested Time: 7 minutes Use this question as a step by step example of how to solve a recursion problem! Also, this is a new question. Please let us know if you have any feedback!

5. The *Gibonacci sequence* is a recursively defined sequence of integers; we denote the  $n$ th Gibonacci number  $g_n$ . The first three terms of the sequence are  $g_0 = 0, g_1 = 1, g_2 = 2$ . For  $n \geq 3$ ,  $g_n$  is defined as the sum of the previous three terms in the sequence.

Complete the function `gib`, which takes in an integer `n` and returns the  $n$ th Gibonacci number,  $g_n$ . Also, identify the three parts of recursive function design as they are used in your solution.

```
def gib(n):  
    """  
    >>> gib(0)  
    0  
    >>> gib(1)  
    1  
    >>> gib(2)  
    2  
    >>> gib(3) # gib(2) + gib(1) + gib(0) = 3  
    3  
    >>> gib(4) # gib(3) + gib(2) + gib(1) = 6  
    6  
    """  
    if _____:  
        return _____  
  
    return _____
```

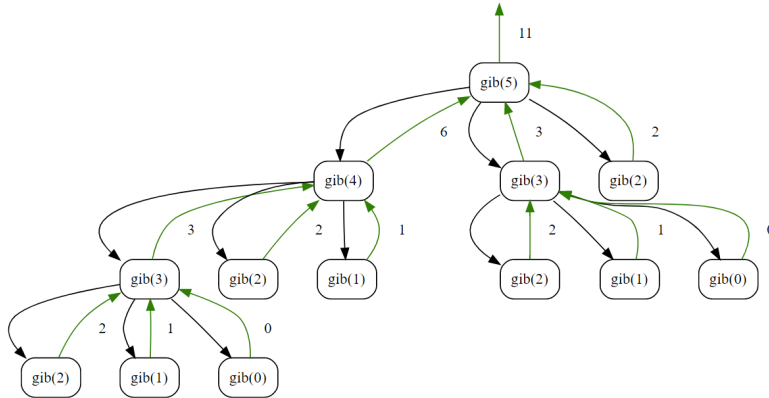
```
def gib(n):  
    if n <= 2:  
        return n  
    return gib(n - 1) + gib(n - 2) + gib(n - 3)
```

- Base case: `if n <= 2: return n`
- Recursive calls: `gib(n - 1), gib(n - 2), gib(n - 3)`
- Solving the larger problem: adding the results of the three recursive calls.
- This problem is very similar to Fibonacci, which students should be familiar with from lecture.
- Ensure that students understand how the minor alteration to the sequence slightly modifies the recursive calls and the base case

6. Including the original call, how many calls are made to `gib` when you evaluate `gib(5)`?

13.

There are a few ways you could determine this. First, you could draw out a call graph that shows all of the calls that are made. <https://tinyurl.com/gibsol>



You could also build things from the ground up using recursive intuition.

- `gib(0)` takes one call because it's a base case.
- `gib(1)` takes one call because it's a base case.
- `gib(2)` takes one call because it's a base case.
- `gib(3)` takes four calls: one for the original call, one for `gib(0)`, one for `gib(1)`, and one for `gib(2)`.
- `gib(4)` takes seven calls: one for the original call, four for `gib(3)`, one for `gib(2)`, and one for `gib(1)`.
- `gib(5)` takes 13 calls: one for the original call, seven for `gib(4)`, four for `gib(3)`, and one for `gib(2)`.

This second method is much faster, especially for higher values of `n`.

In general, I believe that drawing out call graphs can be confusing for students, especially for more difficult recursive problems. The whole idea behind recursion, after all, is that you don't have to worry about the dense tree of calls that result from a single call to a recursive function—you can just trust that the recursive calls you do make return the correct result and don't have to worry about how they specifically got that result. However, I believe that it is useful to do this sort of thing once because it demonstrates how recursion actually works in practice and why the recursion is valid and useful.

When teaching this problem, I recommend first going over the call tree. I would emphasize a few ideas. First, our base cases are at the bottom of the call graph, and every call eventually terminates at one of the base cases. They literally serve as the foundation of the tree. They are the "truth" upon which the function's validity for higher inputs rests. I would also point out that every call that's not a base case has three calls underneath it because each of these calls makes three recursive calls. Hopefully this makes the idea of recursion more concrete for the students and quells perennial concerns that the logic behind recursion is circular.

I would then going over the "smarter" way of doing the problem. I would point out how doing the problem in this way allows us to avoid the repetitive, confusing, and error prone work of considering

every call in the function. I would note that if we tried to do the same thing with `gib(1000)`, the graph would be untenable. The lesson I want them to walk away with is: abstracting away the huge number of calls is a valuable tool when working with recursion, and that they should try to avoid thinking about these calls if at all possible, as it only gets more confusing for more complicated functions.

Suggested Time: 7 minutes

7. Complete the definition for `num_digits`, which takes in a number  $n$  and returns the number of digits it has. Implement the function recursively.

```
def num_digits(n):  
    """Takes in an positive integer and returns the number of  
    digits.  
  
    >>> num_digits(0)  
    1  
    >>> num_digits(1)  
    1  
    >>> num_digits(7)  
    1  
    >>> num_digits(1093)  
    4  
    """  
  
    if n < 10:  
        return 1  
    else:  
        return 1 + num_digits(n // 10)
```

Suggested Time: This question is similar to the Boba Line problem, so it may be helpful to have students try to figure it out on their own first and look back at the other problems for reference.

8. Implement a recursive version of fizzbuzz.

```
def fizzbuzz(n):  
    """Prints the numbers from 1 to n. If the number is divisible by 3, it  
    instead prints 'fizz'. If the number is divisible by 5, it instead  
    prints  
    'buzz'. If the number is divisible by both, it prints 'fizzbuzz'. You  
    must do this recursively!  
  
    >>> fizzbuzz(15)  
    1  
    2  
    fizz  
    4  
    buzz  
    fizz  
    7  
    8  
    fizz  
    buzz  
    11  
    fizz  
    13  
    14  
    fizzbuzz  
    """  
  
    if n == 1:  
        print(n)  
    else:  
        fizzbuzz(n - 1)  
        if n % 3 == 0 and n % 5 == 0:  
            print('fizzbuzz')  
        elif n % 3 == 0:  
            print('fizz')  
        elif n % 5 == 0:  
            print('buzz')  
        else:  
            print(n)
```

It may be beneficial to reiterate the recursive leap of faith! Hypothetically, if fizzbuzz works for n, what would fizzbuzz(n - 1) output?